

Experiment :9

Aim

To implement a Recurrent Neural Network (RNN) / Long Short-Term Memory (LSTM) model for time series data using the Air Quality dataset.

Data Source

Air Quality - UCI Machine Learning Repository

The Air Quality dataset is a time-series dataset used for air pollution analysis and prediction.

<https://archive.ics.uci.edu/dataset/360/air+quality>

Key Details

- **Instances:** 9,358 records
- **Features:** 15 attributes
- **Type:** Multivariate, Time-series
- **Duration:** March 2004 – February 2005 (1 year)

Data Description

- Data collected from 5 gas sensors in an Italian city
- Measures pollutant concentrations such as:
 - CO (Carbon Monoxide)
 - Benzene (C₆H₆)
 - NO_x and NO₂
- Includes both sensor readings and ground truth values from a reference analyzer

Theory

Recurrent Neural Network (RNN)

A Recurrent Neural Network (RNN) is a type of neural network designed to process sequential data. Unlike feedforward neural networks, RNNs maintain a hidden state that

captures information from previous time steps, enabling them to model temporal dependencies.

RNNs are widely used in:

- Time series forecasting
- Natural Language Processing (NLP)
- Speech recognition

Working of RNN

In an RNN, the output at a given time step depends on both the current input and the previous hidden state.

Mathematically:

$$h_t = f(Wx_t + Uh_{t-1})$$

Where:

- h_t = hidden state at time t
- x_t = input at time t
- W, U = weight matrices

Limitation of RNN

A major limitation of RNN is the **vanishing gradient problem**, which makes it difficult to learn long-term dependencies.

Introduction to LSTM

Long Short-Term Memory (LSTM) is an advanced type of RNN designed to overcome the vanishing gradient problem. It is capable of learning long-term dependencies using a memory cell and gating mechanisms.

LSTM Architecture

An LSTM network consists of the following components:

- **Forget Gate:** Decides which information to discard from the cell state
- **Input Gate:** Determines which new information to add
- **Output Gate:** Controls what information is passed to the next hidden state

The **cell state** acts as long-term memory, allowing information to persist across many time steps.

RNN/LSTM for Time Series Data

Applications

- Stock price prediction
- Weather forecasting
- Demand forecasting

Steps Involved

1. Collect sequential data
2. Normalize the data
3. Convert data into sequences using sliding window method
4. Train the RNN or LSTM model
5. Predict future values

Example: Using past 10 time steps to predict the next value.

RNN/LSTM for Text Data

Applications

- Sentiment analysis
- Machine translation
- Text generation

Steps Involved

1. Text preprocessing (tokenization and padding)
2. Convert text into numerical form (word embeddings)
3. Input sequences into the model
4. Generate predictions (class or next word)

Example: Predicting sentiment from a sentence.

Advantages

- Effective for sequential and time-series data
- Captures temporal relationships

- LSTM handles long-term dependencies better than standard RNN

Limitations

- RNN suffers from vanishing gradient problem
- LSTM models are computationally intensive
- Requires large datasets for better performance

Applications

- Speech recognition
- Chatbots
- Time series forecasting
- Language translation

Code:

Install libraries

```
!pip install pandas numpy matplotlib scikit-learn tensorflow
```

Imports

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
from sklearn.metrics import mean_squared_error, mean_absolute_error
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import LSTM, Dense
```

Load dataset

```
url =
```

```
"https://archive.ics.uci.edu/ml/machine-learning-databases/00360/AirQualityUCI.zip"
```

```
import zipfile, requests, io
```

```
r = requests.get(url)
```

```
z = zipfile.ZipFile(io.BytesIO(r.content))
```

```
z.extractall()
```

```
df = pd.read_csv("AirQualityUCI.csv", sep=';', decimal=',')
```

Preprocessing

```

df = df.iloc[:, :-2]
df.replace(-200, np.nan, inplace=True)
df.fillna(method='ffill', inplace=True)

# Fix datetime
df['Time'] = df['Time'].str.replace('.', ':', regex=False)
df['Datetime'] = pd.to_datetime(df['Date'] + ' ' + df['Time'], format='%d/%m/%Y
%H:%M:%S', errors='coerce')
df.dropna(subset=['Datetime'], inplace=True)
df.set_index('Datetime', inplace=True)

# Select multiple features (like your output)
features = ['PT08.S1(CO)', 'PT08.S2(NMHC)', 'PT08.S3(NOx)', 'PT08.S4(NO2)',
'PT08.S5(O3)']
data = df[features].values

# Normalize
scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data)

# Create sequences
def create_dataset(dataset, time_step=20):
    X, Y = [], []
    for i in range(len(dataset) - time_step - 1):
        X.append(dataset[i:(i + time_step)])
        Y.append(dataset[i + time_step])
    return np.array(X), np.array(Y)

time_step = 20
X, y = create_dataset(data_scaled, time_step)

# Train-test split
split = int(len(X) * 0.8)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# Build LSTM model
model = Sequential()
model.add(LSTM(64, return_sequences=True, input_shape=(time_step, X.shape[2])))
model.add(LSTM(64))

```

```

model.add(Dense(X.shape[2]))

model.compile(optimizer='adam', loss='mse', metrics=['mae'])

# Train
history = model.fit(X_train, y_train, epochs=15, batch_size=32,
                    validation_data=(X_test, y_test), verbose=1)

# Predict
pred = model.predict(X_test)

# Inverse scaling
pred_inv = scaler.inverse_transform(pred)
y_test_inv = scaler.inverse_transform(y_test)

plt.figure(figsize=(12, 18))

for i in range(len(features)):
    plt.subplot(len(features), 1, i+1)
    plt.plot(y_test_inv[:, i], label="Actual")
    plt.plot(pred_inv[:, i], label="Predicted")
    plt.title(f"LSTM Prediction: {features[i]}")
    plt.legend()

plt.tight_layout()
plt.show()

residuals = y_test_inv - pred_inv

plt.figure(figsize=(12, 18))

for i in range(len(features)):
    # Residual over time
    plt.subplot(len(features), 2, 2*i+1)
    plt.plot(residuals[:, i])
    plt.axhline(0, color='red')
    plt.title(f"Residuals Over Time: {features[i]}")

    # Histogram
    plt.subplot(len(features), 2, 2*i+2)

```

```

plt.hist(residuals[:, i], bins=50)
plt.title(f'Residual Distribution: {features[i]}')

plt.tight_layout()
plt.show()

plt.figure(figsize=(12,5))

# Loss
plt.subplot(1,2,1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("LSTM Training Loss (MSE)")
plt.legend()

# MAE
plt.subplot(1,2,2)
plt.plot(history.history['mae'], label='Train MAE')
plt.plot(history.history['val_mae'], label='Validation MAE')
plt.title("LSTM Training MAE")
plt.legend()

plt.show()

for i, feature in enumerate(features):
    rmse = np.sqrt(mean_squared_error(y_test_inv[:, i], pred_inv[:, i]))
    mae = mean_absolute_error(y_test_inv[:, i], pred_inv[:, i])
    print(f'{feature} -> RMSE: {rmse:.4f}, MAE: {mae:.4f}')

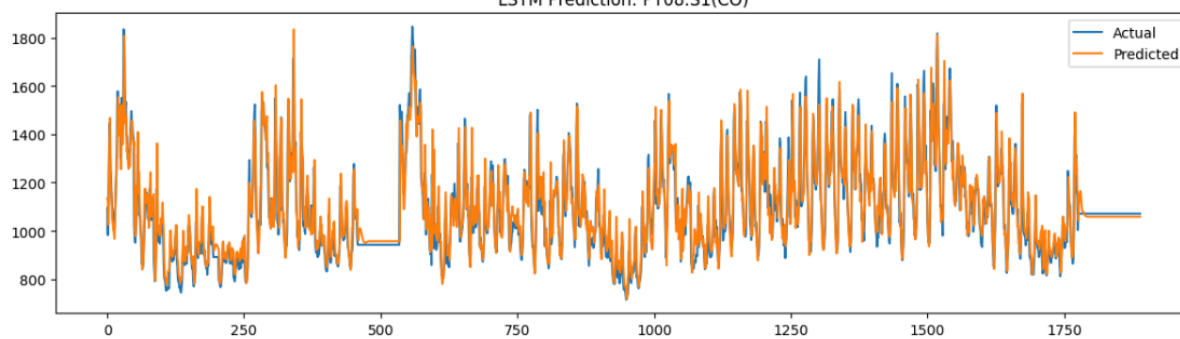
```

Output:

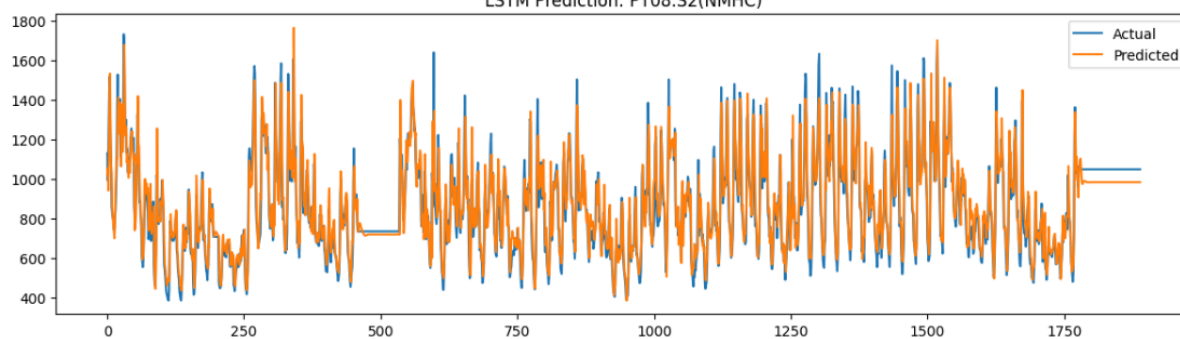
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.19.0)
Requirement already satisfied: python-dateutil<2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.7.0)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (5.29.6)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.1.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.80.0)
Requirement already satisfied: tensorboard>=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.13.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.16.0)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse==1.6.0->tensorflow) (0.46.3)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.19.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.4.6)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2026.2.25)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard>=2.19.0->tensorflow) (3.10.2)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard>=2.19.0->tensorflow) (0.7.2)
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard>=2.19.0->tensorflow) (3.1.7)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboard>=2.19.0->tensorflow) (3.0.3)

... 237/237 ————— 9s 24ms/step - loss: 0.0151 - mae: 0.0921 - val_loss: 0.0077 - val_mae: 0.0695
Epoch 2/15
237/237 ————— 7s 29ms/step - loss: 0.0053 - mae: 0.0543 - val_loss: 0.0042 - val_mae: 0.0491
Epoch 3/15
237/237 ————— 9s 26ms/step - loss: 0.0043 - mae: 0.0484 - val_loss: 0.0037 - val_mae: 0.0441
Epoch 4/15
237/237 ————— 6s 24ms/step - loss: 0.0041 - mae: 0.0473 - val_loss: 0.0035 - val_mae: 0.0426
Epoch 5/15
237/237 ————— 11s 29ms/step - loss: 0.0038 - mae: 0.0455 - val_loss: 0.0032 - val_mae: 0.0408
Epoch 6/15
237/237 ————— 10s 26ms/step - loss: 0.0038 - mae: 0.0452 - val_loss: 0.0034 - val_mae: 0.0417
Epoch 7/15
237/237 ————— 6s 25ms/step - loss: 0.0037 - mae: 0.0448 - val_loss: 0.0032 - val_mae: 0.0401
Epoch 8/15
237/237 ————— 5s 22ms/step - loss: 0.0036 - mae: 0.0441 - val_loss: 0.0032 - val_mae: 0.0408
Epoch 9/15
237/237 ————— 7s 28ms/step - loss: 0.0035 - mae: 0.0437 - val_loss: 0.0031 - val_mae: 0.0398
Epoch 10/15
237/237 ————— 5s 23ms/step - loss: 0.0034 - mae: 0.0429 - val_loss: 0.0035 - val_mae: 0.0432
Epoch 11/15
237/237 ————— 10s 22ms/step - loss: 0.0034 - mae: 0.0425 - val_loss: 0.0029 - val_mae: 0.0375
Epoch 12/15
237/237 ————— 6s 27ms/step - loss: 0.0033 - mae: 0.0419 - val_loss: 0.0031 - val_mae: 0.0395
Epoch 13/15
237/237 ————— 9s 23ms/step - loss: 0.0032 - mae: 0.0413 - val_loss: 0.0029 - val_mae: 0.0381
Epoch 14/15
237/237 ————— 10s 22ms/step - loss: 0.0031 - mae: 0.0408 - val_loss: 0.0028 - val_mae: 0.0370
Epoch 15/15
237/237 ————— 7s 28ms/step - loss: 0.0030 - mae: 0.0400 - val_loss: 0.0027 - val_mae: 0.0369
60/60 ————— 1s 12ms/step

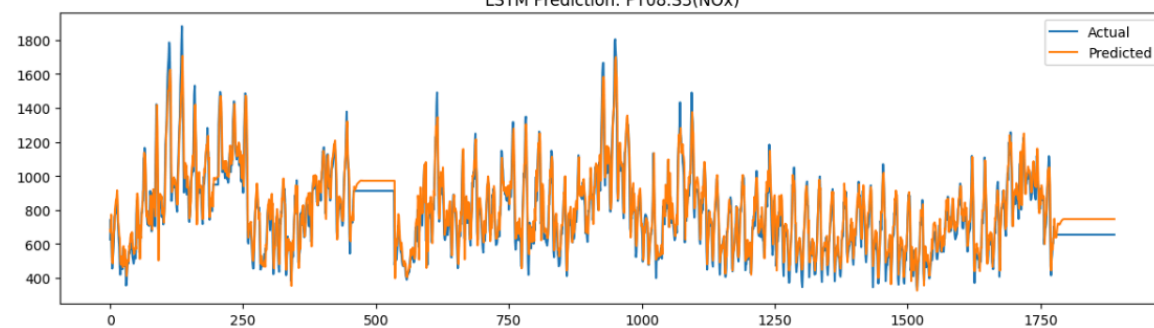
LSTM Prediction: PT08.S1(CO)



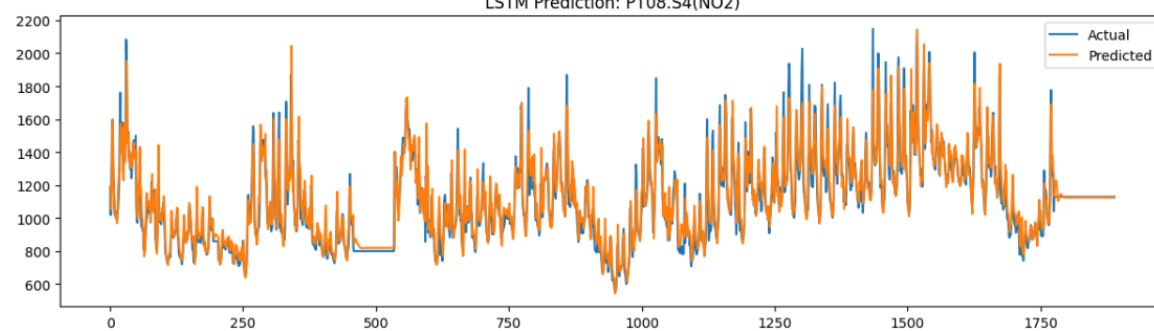
LSTM Prediction: PT08.S2(NMHC)

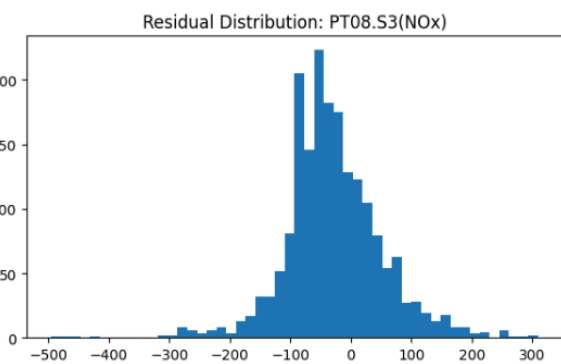
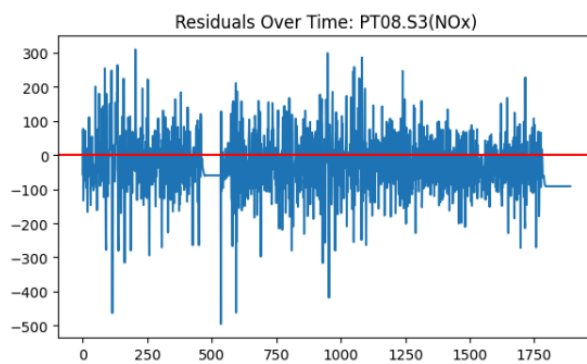
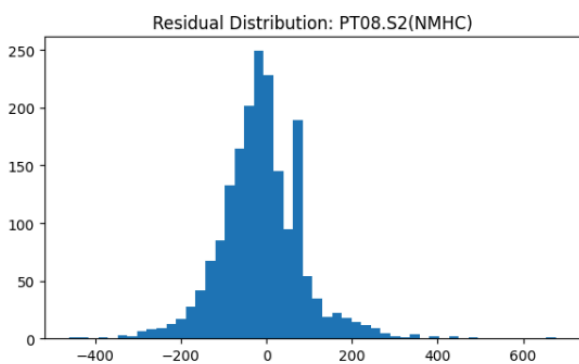
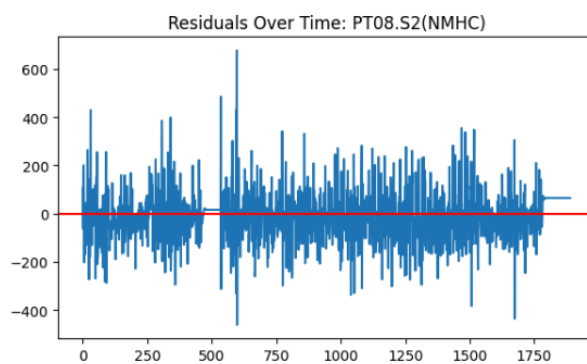
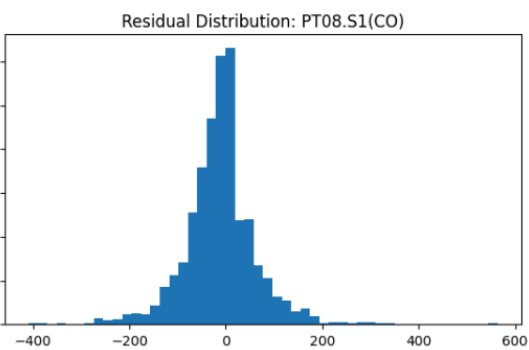
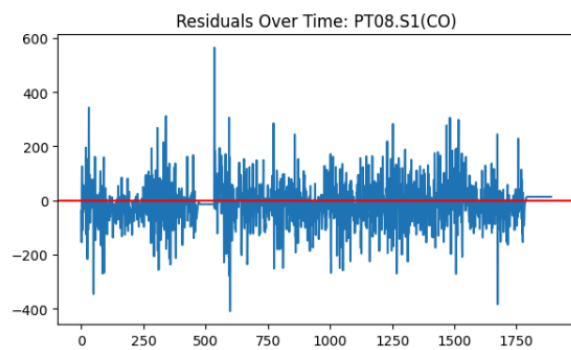
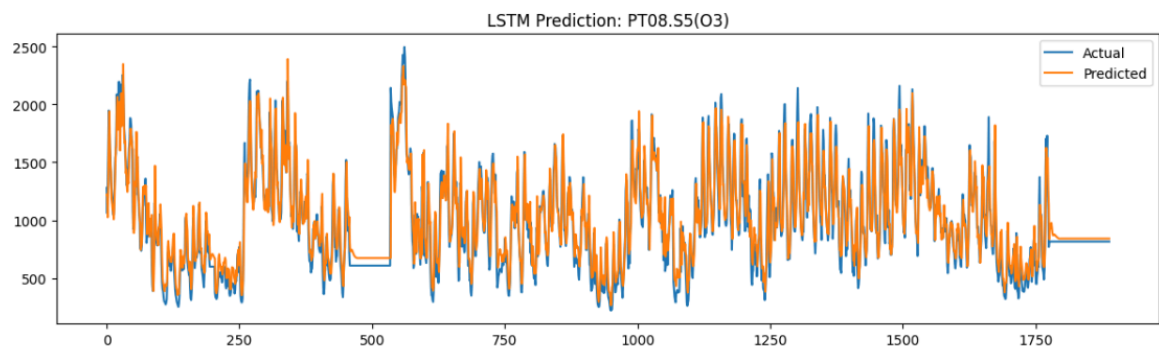


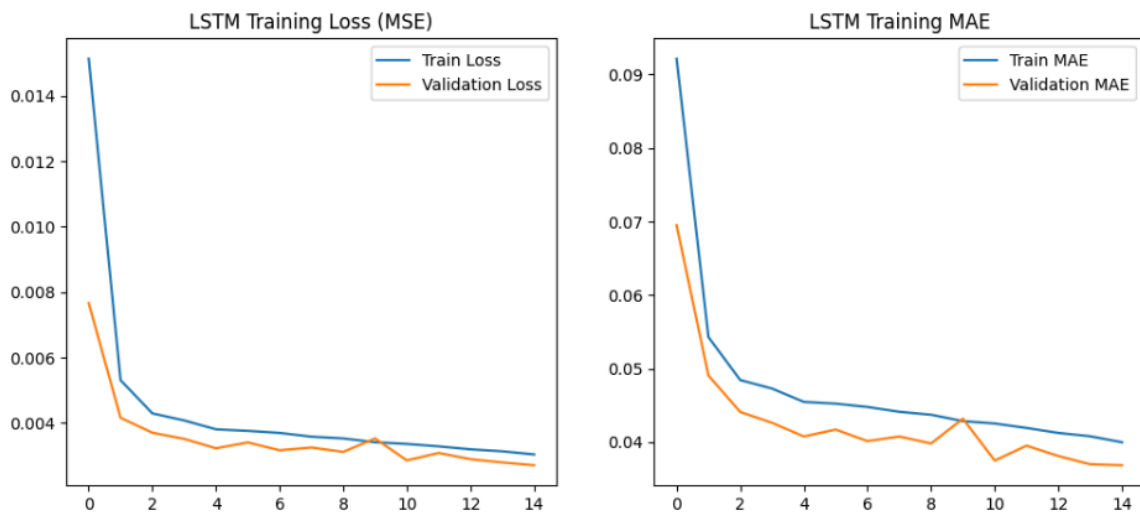
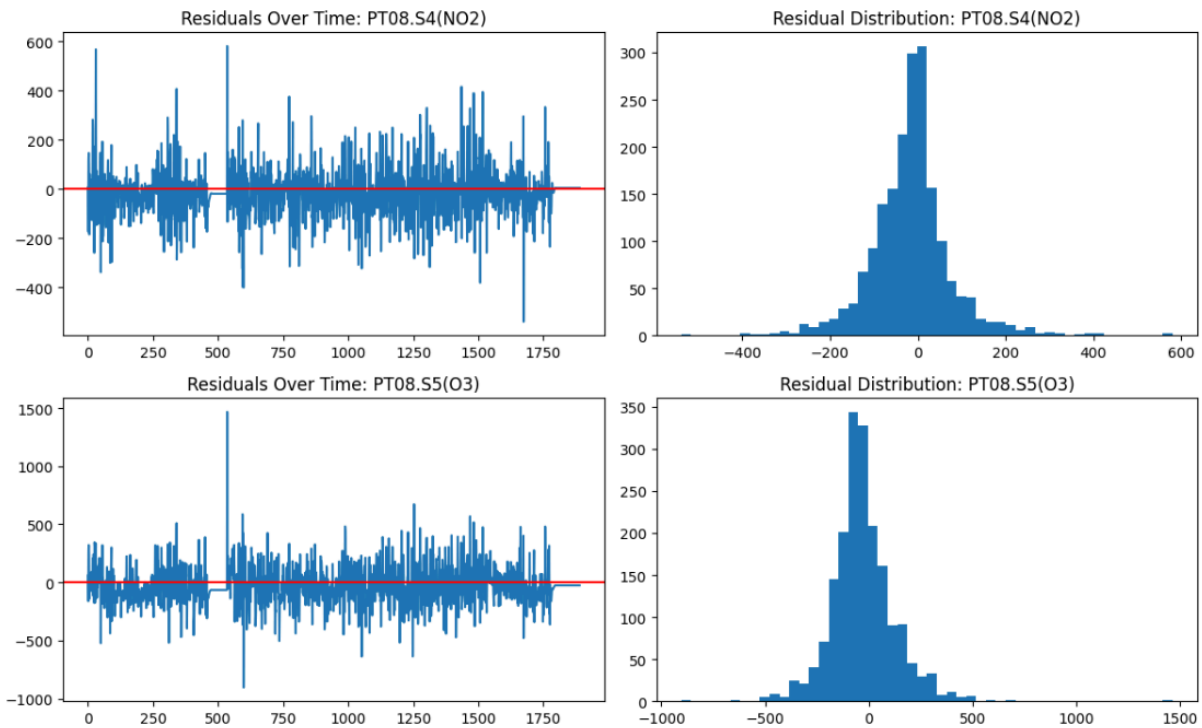
LSTM Prediction: PT08.S3(NOx)



LSTM Prediction: PT08.S4(NO2)







PT08.S1(CO) -> RMSE: 76.0532, MAE: 53.2285
PT08.S2(NMHC) -> RMSE: 98.4120, MAE: 71.8393
PT08.S3(NOx) -> RMSE: 86.5429, MAE: 66.7722
PT08.S4(NO2) -> RMSE: 93.3881, MAE: 64.2958
PT08.S5(O3) -> RMSE: 155.4701, MAE: 114.3610

Conclusion:

In this experiment, an LSTM model was implemented on the Air Quality time-series dataset. After preprocessing and sequence creation, the model successfully learned temporal patterns and predicted future values with good accuracy. The results show that LSTM is effective for time-series forecasting as it can capture long-term dependencies better than a basic RNN.